PROYECTO DE AUTOMATIZACIÓN CON ANSIBLE: SOLUCIÓN DE INFRAESTRUCTURA EN ENTORNO REAL

Cristóbal Suárez Abad

OPTATIVA - 2º ASIR

Índice

Descripción del problema. 2
Explicación de la arquitectura diseñada. 2
Instrucciones de ejecución. 3
Modificación del Dockerfile de “nodes”. 3
Modificación de docker-compose.yml 3
Gestión de Inventario y Conectividad 4
Implementación de Seguridad y Acceso (SSH) 5
Lógica de Automatización (Playbooks) 9
Definición de Variables 9
Tareas del Rol 10
Crear el Handler 13
El Playbook Principal 14
Instalar el Rol de Galaxy 17
Cómo lanzar el proyecto 19
Levantar el escenario (Docker) 19
Configuración inicial de acceso (SSH) 19
Ejecución del Hardening Global 19
Despliegue de Servicios (Ansible Galaxy) 19
Verificación 20

Descripción del problema.

Hardening de Servidores: Automatizar el cierre de puertos, actualización de parches y creación de usuarios con permisos limitados.

Explicación de la arquitectura diseñada.

Se aprovechó en Docker del laboratorio Ansible → "LaboratorioAnsible.zip" del Tema 4.

Por lo tanto, la arquitectura es:

Un nodo de control (ansible-control) y tres nodos gestionados (node1, node2, node3) interconectados en una red privada bridge de Docker

Instrucciones de ejecución.

Modificación del Dockerfile de “nodes”.

Debemos añadir las siguientes líneas para evitar problemas

Debes añadir sudo al listado de paquetes a instalar

RUN apt-get update && apt-get install -y openssh-server python3 sudo

Configura al usuario ansible en el grupo sudo

RUN usermod -aG sudo ansible

Se modificó la imagen base de los nodos gestionados mediante Dockerfile para incluir el paquete sudo y añadir al usuario de gestión (ansible) al grupo de administradores.

Esta decisión técnica es fundamental para cumplir con el principio de **mínimo privilegio**, permitiendo que Ansible opere de forma remota como un usuario sin privilegios y solo escale a permisos de **superusuario (become)** en las tareas estrictamente necesarias, como la configuración del firewall o la actualización de parches de seguridad.

Por defecto, las imágenes de Docker de Ubuntu son tan minimalistas que **no traen el comando sudo** instalado. Cuando Ansible intentaba ejecutar una tarea con become: yes, le enviaba al nodo la orden, pero el nodo no respondía y la orden fallaba.

Modificación de docker-compose.yml

Debemos añadir las siguientes líneas en cada uno de los nodos gestionados:

```
privileged: true
cap_add:
 - NET_ADMIN
```

Al intentar automatizar el firewall (UFW), se encontró una limitación de seguridad de Docker. Los contenedores no poseen por defecto capacidades de administración de red (NET_ADMIN). Se resolvió elevando los privilegios de los contenedores en el archivo docker-compose.yml para permitir que Ansible manipule las reglas de iptables.

Gestión de Inventario y Conectividad

Inventario: *Crear y mantener archivos de inventario organizados por grupos.*

Se creó un archivo inventory organizando los hosts por grupos lógicos:

[webservers], [dbservers] y [testing].

Se implementó un grupo padre [produccion:children] que agrupa a los servidores web y de base de datos.

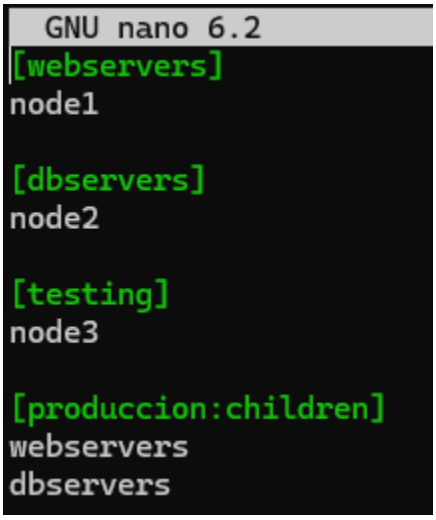
De esta manera se cumple con el requisito de gestión de inventario organizado, permitiendo lanzar tareas a grupos específicos o a todo el entorno de producción simultáneamente.

[webservers]
node1

[dbservers]
node2

[testing]
node3

[produccion:children]
webservers
dbservers



Implementación de Seguridad y Acceso (SSH)

Creamos un archivo en “groups_vars” → nano **group_vars/all.yml**

Donde indicaremos credenciales de usuario y contraseñas de “root”.

ansible_user: ansible

#ansible_password: password

ansible_become_pass: password

```
  GNU nano 6.2                                                    group
---
ansible_user: ansible
#ansible_password: password
ansible_become_pass: password # Necesario para cuando usemos 'become'
```

NOTA: “ansible_password” está comentado porque en este punto ya había configurado el acceso por claves RSA. De esta manera se eliminó la contraseña de texto plano del inventario una vez establecida la confianza SSH.

- Generamos las claves RSA:

ssh-keygen -t rsa -b 4096 -N "" -f ~/.ssh/id_rsa

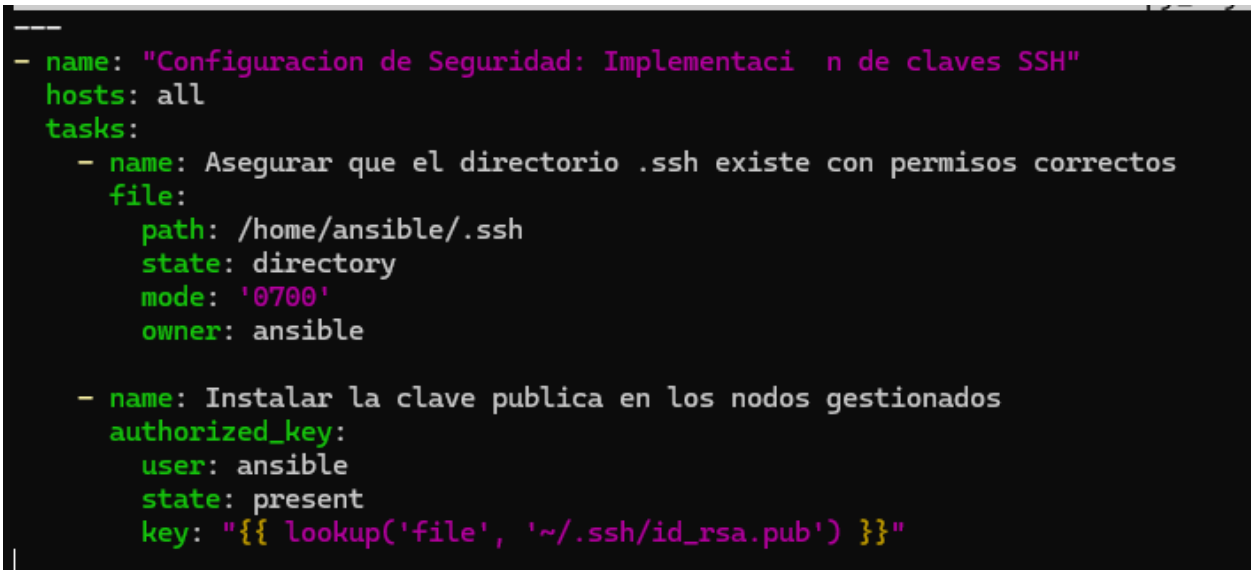
```
root@control:/ansible/proyecto2# ssh-keygen -t rsa -b 4096 -N "" -f ~/.ssh/id_rsa
Generating public/private rsa key pair.
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:V+4sypra/S3UKxBSM0TmwXeZIF/qMl857kozBC67QEc root@control
The key's randomart image is:
+---[RSA 4096]----+
|       +* ...o   |
|       o=+.o+    |
|      E.o+o..    |
|     ....o o .   |
|    . o.S.+.=    |
|   . . o.*.=..   |
|    . .  o* +.   |
|     o = oo*.    |
|    ..=.+.o+o    |
+----[SHA256]-----+
```

Creamos el Playbook → copy_keys.yml

Lo usaremos para distribuir las claves públicas.

```
---
- name: "Configuracion de Seguridad: Implementaci  n de claves SSH"
  hosts: all
  tasks:
    - name: Asegurar que el directorio .ssh existe con permisos correctos
      file:
        path: /home/ansible/.ssh
        state: directory
        mode: '0700'
        owner: ansible

    - name: Instalar la clave publica en los nodos gestionados
      authorized_key:
        user: ansible
        state: present
        key: "{{ lookup('file', '~/.ssh/id_rsa.pub') }}"
```



Lo ejecutamos: **ansible-playbook copy_keys.yml**

```
root@control:/ansible/proyecto2# ansible-playbook copy_keys.yml

PLAY [Configuración de Seguridad: Implementación de claves SSH] ***************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [node1]
ok: [node2]
ok: [node3]

TASK [Asegurar que el directorio .ssh existe con permisos correctos] ***********************************************
changed: [node1]
changed: [node3]
changed: [node2]

TASK [Instalar la clave publica en los nodos gestionados] **********************************************************
changed: [node3]
changed: [node1]
changed: [node2]

PLAY RECAP *********************************************************************************************************
node1                      : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
node2                      : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
node3                      : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

En el archivo "**group_vars/all.yml**" comentamos la clave para el usuario "ansible", así sabremos que funcionan las claves RSA.

Ahora comprobamos la conexión con los nodos: **ansible all -m ping**

```
root@control:/ansible/proyecto2# ansible all -m ping
node3 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
node1 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
node2 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

Casi se me olvida, antes de todo esto debemos tener un archivo "**ansible.cfg**" con la siguiente configuración:

[defaults]
inventory = inventory # Indica el archivo con el inventario.
host_key_checking = False # Desactiva la verificación de la clave SSH de los hosts remotos. Así no preguntará por la confirmación y se conectará automáticamente.
remote_user = ansible # Define el usuario por defecto

```
[defaults]
inventory = inventory
host_key_checking = False
remote_user = ansible
```

Con este sistema de Claves RSA (clave pública y privada) se garantiza una comunicación segura y se cumple con el estándar de autenticación sin contraseñas que normalmente se requiere en infraestructuras IT profesionales.

En esta primera parte hemos podido comprobar que el nodo control resuelve perfectamente los nombres de los nodos gestionados y que las credenciales y conexión por SSH funcionan correctamente.

Lógica de Automatización (Playbooks)

Vamos a crear un "Rol" llamado "common" con el cual estandarizaremos la seguridad en todos los nodos.

Dentro de la carpeta de "proyectos2" creamos el directorio "roles". Y dentro de él ejecutamos: **ansible-galaxy role init common**

```
root@control:/ansible/proyecto2/roles# ansible-galaxy role init common
- Role common was created successfully
root@control:/ansible/proyecto2/roles# ls -l
total 4
drwxr-xr-x 8 root root 4096 Mar 22 15:16 common
```

Definición de Variables

Primero, definiremos qué paquetes de seguridad queremos instalar. Esto cumple con el requisito de **Gestión de Variables**.

nano **roles/common/vars/main.yml**

```
---
# Paquetes esenciales para auditoría y seguridad
seguridad_paquetes:
 - ufw
 - fail2ban
 - htop
 - curl
```

```
---
# Paquetes esenciales para auditoria y seguridad
seguridad_paquetes:
  - ufw
  - fail2ban
  - htop
  - curl
```

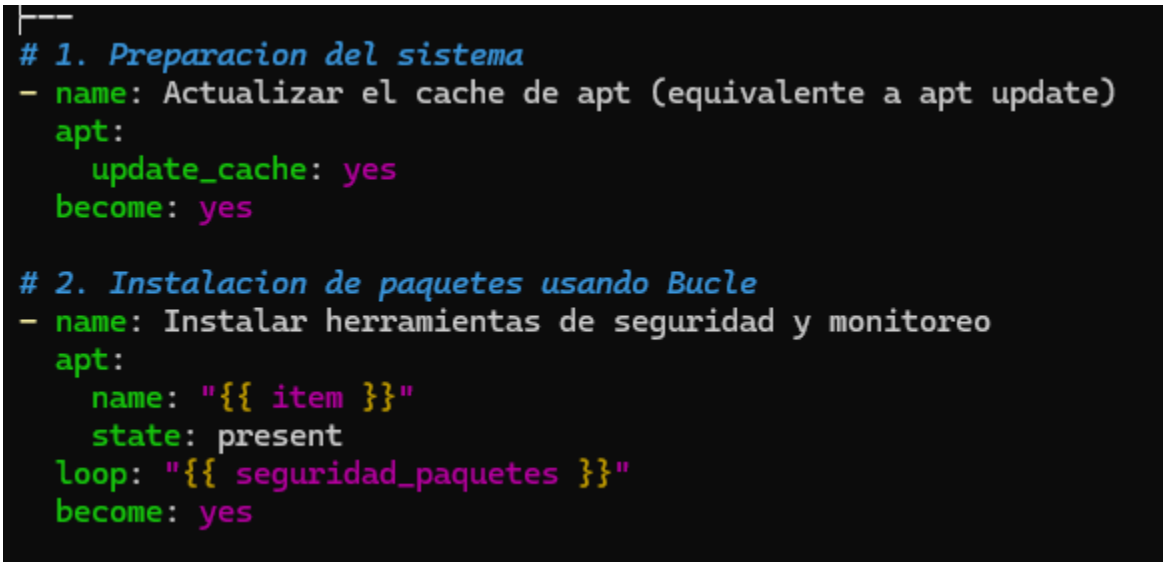
Tareas del Rol

Aquí implementaremos la lógica usando **bucles** (loop) y **escalada de privilegios** (become).

nano **roles/common/tasks/main.yml**

1. Preparacion del sistema
- name: Actualizar el cache de apt (equivalente a apt update)
  apt:
    update_cache: yes
  become: yes

2. Instalacion de paquetes usando Bucle
- name: Instalar herramientas de seguridad y monitoreo
  apt:
    name: "{{ item }}"
    state: present
  loop: "{{ seguridad_paquetes }}"
  become: yes



3. Configuracion de Red (Hardening de Firewall)
IMPORTANTE: El "allow 22" debe ir SIEMPRE antes de "enabled"
- name: Permitir trafico SSH (Puerto 22) para no perder la conexion
 ufw:
  rule: allow
  port: '22'
  proto: tcp
 become: yes

- name: Configurar politica por defecto y habilitar UFW
 ufw:
  state: enabled
  policy: deny
  direction: incoming
 become: yes
 when: ansible_distribution == "Ubuntu" # Uso de condicional

```yaml
# 3. Configuracion de Red (Hardening de Firewall)
# IMPORTANTE: El "allow 22" debe ir SIEMPRE antes de "enabled"
- name: Permitir trafico SSH (Puerto 22) para no perder la conexion
  ufw:
    rule: allow
    port: '22'
    proto: tcp
  become: yes

- name: Configurar politica por defecto y habilitar UFW
  ufw:
    state: enabled
    policy: deny
    direction: incoming
  become: yes
  when: ansible_distribution == "Ubuntu"  # Uso de condicional
```

```
# 4. Hardening de SSH
- name: Deshabilitar el acceso root por SSH
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^PermitRootLogin'
    line: 'PermitRootLogin no'
  become: yes
  notify: Reiniciar SSH  # Uso de Handler

# 5. Verificacion y Registro
- name: Verificar estado del firewall
  command: ufw status
  register: ufw_result  # Uso de Register
  become: yes

- name: Mostrar el estado del firewall en la consola
  debug:
    msg: "El estado del firewall es: {{ ufw_result.stdout }}"
```


```yaml
# 4. Hardening de SSH
- name: Deshabilitar el acceso root por SSH
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^PermitRootLogin'
    line: 'PermitRootLogin no'
  become: yes
  notify: Reiniciar SSH  # Uso de Handler

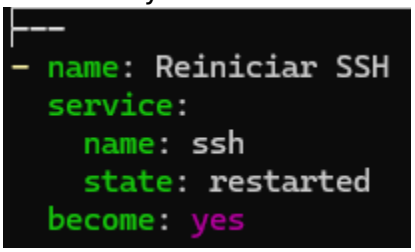
# 5. Verificacion y Registro
- name: Verificar estado del firewall
  command: ufw status
  register: ufw_result  # Uso de Register
  become: yes

- name: Mostrar el estado del firewall en la consola
  debug:
    msg: "El estado del firewall es: {{ ufw_result.stdout }}"
```

Crear el Handler

nano **roles/common/handlers/main.yml**

```
---
- name: Reiniciar SSH
  service:
    name: ssh
    state: restarted
  become: yes
```



El Playbook Principal

Este es el archivo que "llama" a los roles y los aplica a tus grupos de inventario. Lo creamos en el directorio principal:

nano **site.yml**

```
---
- name: "Proyecto FDA: Hardening Global de Infraestructura"
  hosts: all
  roles:
    - common
```

Ahora lo ejecutamos con: **ansible-playbook site.yml**

```
root@control:/ansible/proyecto2# ansible-playbook site.yml

PLAY [Proyecto FDA: Hardening Global de Infraestructura] **********************

TASK [Gathering Facts] ********************************************************
ok: [node3]
ok: [node2]
ok: [node1]

TASK [common : Actualizar el cache de apt (equivalente a apt update)] *********
[WARNING]: Updating cache and auto-installing missing dependency: python3-apt
ok: [node3]
ok: [node1]
ok: [node2]

TASK [common : Instalar herramientas de seguridad y monitoreo] ****************
changed: [node3] => (item=ufw)
changed: [node1] => (item=ufw)
changed: [node2] => (item=ufw)
changed: [node3] => (item=fail2ban)
changed: [node1] => (item=fail2ban)
changed: [node2] => (item=fail2ban)
changed: [node1] => (item=htop)
changed: [node3] => (item=htop)
changed: [node2] => (item=htop)
changed: [node1] => (item=curl)
changed: [node3] => (item=curl)
changed: [node2] => (item=curl)
```

```
TASK [common : Instalar herramientas de seguridad y monitoreo] ****
changed: [node2] => (item=ufw)
changed: [node3] => (item=ufw)
changed: [node1] => (item=ufw)
changed: [node1] => (item=fail2ban)
changed: [node2] => (item=fail2ban)
changed: [node3] => (item=fail2ban)
changed: [node1] => (item=htop)
changed: [node2] => (item=htop)
changed: [node3] => (item=htop)
changed: [node3] => (item=curl)
changed: [node1] => (item=curl)
changed: [node2] => (item=curl)

TASK [common : Configurar politica por defecto de UFW (Firewall)] *
changed: [node2]
changed: [node3]
changed: [node1]

TASK [common : Deshabilitar el acceso root por SSH (Hardening)] ****
changed: [node2]
changed: [node3]
changed: [node1]

TASK [common : Verificar estado del firewall] **********************
changed: [node3]
changed: [node1]
changed: [node2]
```

```
TASK [common : Mostrar el estado del firewall en la consola] ***********************************************************
ok: [node3] => {
    "msg": "El estado del firewall es: Status: active\n\nTo                         Action      From\n--                         ------      ----\n
22/tcp                     ALLOW       Anywhere                  \n22/tcp (v6)               ALLOW       Anywhere (v6)             "
}
ok: [node1] => {
    "msg": "El estado del firewall es: Status: active\n\nTo                         Action      From\n--                         ------      ----\n
22/tcp                     ALLOW       Anywhere                  \n22/tcp (v6)               ALLOW       Anywhere (v6)             "
}
ok: [node2] => {
    "msg": "El estado del firewall es: Status: active\n\nTo                         Action      From\n--                         ------      ----\n
22/tcp                     ALLOW       Anywhere                  \n22/tcp (v6)               ALLOW       Anywhere (v6)             "
}

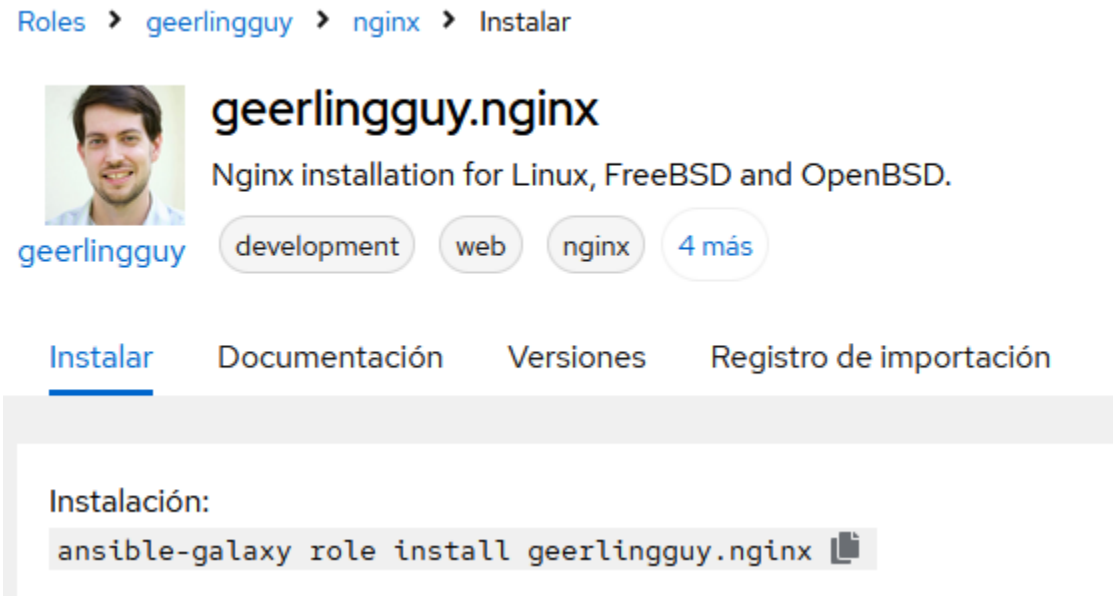
PLAY RECAP *************************************************************************************************************
```

```
PLAY RECAP *************************************************************************************************************
node1                      : ok=8    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
node2                      : ok=8    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
node3                      : ok=8    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Instalar el Rol de Galaxy

Utilizaremos un rol de Galaxy para instalar Nginx en los servidores pertenecientes a “webservers”:

Vamos a usar: https://galaxy.ansible.com/ui/standalone/roles/geerlingguy/nginx/install/



Para que se instale en el directorio “roles” usamos:

ansible-galaxy install geerlingguy.nginx -p ./roles

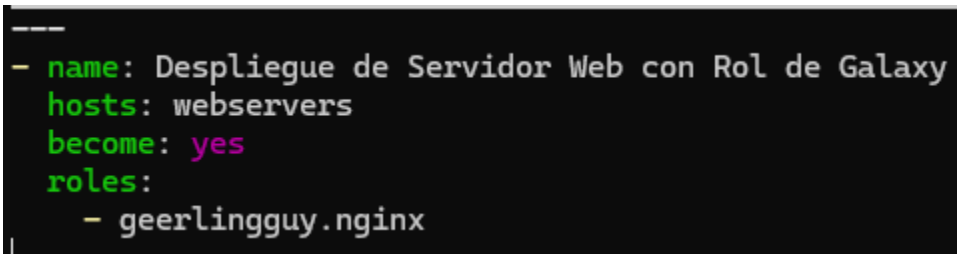
```
root@control:/ansible/proyecto2# ansible-galaxy install geerlingguy.nginx -p ./roles
Starting galaxy role install process
- downloading role 'nginx', owned by geerlingguy
- downloading role from https://github.com/geerlingguy/ansible-role-nginx/archive/3.3.0.tar.gz
- extracting geerlingguy.nginx to /ansible/proyecto2/roles/geerlingguy.nginx
- geerlingguy.nginx (3.3.0) was installed successfully
```

```
root@control:/ansible/proyecto2# ls -l roles/
total 8
drwxr-xr-x  8 root root 4096 Mar 22 15:16 common
drwxr-xr-x 10 root root 4096 Mar 22 17:24 geerlingguy.nginx
root@control:/ansible/proyecto2# ls -l roles/geerlingguy.nginx/
total 44
-rw-rw-r-- 1 root root  1080 Dec  2 16:05 LICENSE
-rw-rw-r-- 1 root root 11874 Dec  2 16:05 README.md
drwxr-xr-x 2 root root  4096 Mar 22 17:24 defaults
drwxr-xr-x 2 root root  4096 Mar 22 17:24 handlers
drwxr-xr-x 2 root root  4096 Mar 22 17:24 meta
drwxr-xr-x 3 root root  4096 Mar 22 17:24 molecule
drwxr-xr-x 2 root root  4096 Mar 22 17:24 tasks
drwxr-xr-x 2 root root  4096 Mar 22 17:24 templates
drwxr-xr-x 2 root root  4096 Mar 22 17:24 vars
root@control:/ansible/proyecto2#
```

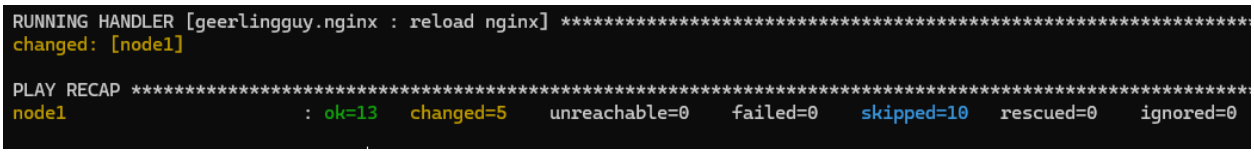
Ahora creamos el Playbook en el directorio principal:

nano **deploy_web.yml**

```
---
- name: Despliegue de Servidor Web con Rol de Galaxy
  hosts: webservers
  become: yes
  roles:
    - geerlingguy.nginx
```



Lo ejecutamos con: **ansible-playbook deploy_web.yml**



Ahora los servidores Web tienen Nginx instalado.

Cómo lanzar el proyecto

Levantar el escenario (Docker)

Desde la máquina anfitriona (Windows/Linux), sitúate en la carpeta del proyecto y ejecuta:

docker compose up -d

Configuración inicial de acceso (SSH)

Entra en el contenedor de control:

docker exec -it ansible-control bash

cd /ansible/proyecto2

Para establecer la confianza entre el control y los nodos sin usar contraseñas manualmente, ejecuta el playbook de intercambio de llaves:

ansible-playbook copy_keys.yml

Ejecución del Hardening Global

Aplica las políticas de seguridad (Firewall UFW, deshabilitar acceso Root, instalación de herramientas) ejecutando el playbook principal:

ansible-playbook site.yml

Despliegue de Servicios (Ansible Galaxy)

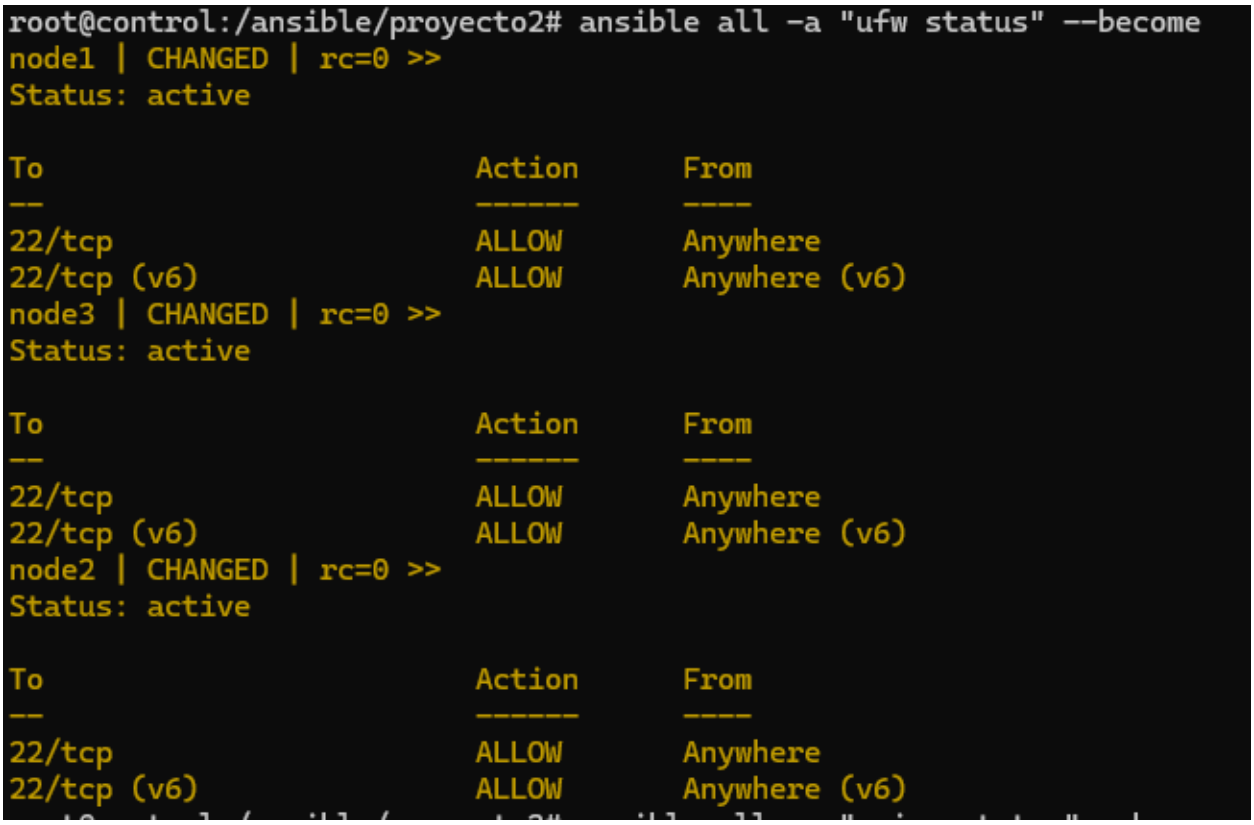
Para completar la configuración del servidor web profesional (Nginx), lanza el playbook de modularidad:

ansible-playbook deploy_web.yml

Verificación

Una vez finalizado, puedes comprobar el estado del firewall en todos los nodos con:

ansible all -a "ufw status" --become



Para los servidores web:

ansible webservers -a "service nginx status" –become

```
root@control:/ansible/proyecto2# ansible webservers -a "service nginx status" --become
[WARNING]: Consider using the service module rather than running 'service'.  If you need
add 'warn: false' to this command task or set 'command_warnings=False' in ansible.cfg to
node1 | CHANGED | rc=0 >>
 * nginx is running
```

o

ansible webservers -a "curl -I localhost" --become

```
root@control:/ansible/proyecto2# ansible webservers -a "curl -I localhost" --become
[WARNING]: Consider using the get_url or uri module rather than running 'curl'.  If you need to use comm
you can add 'warn: false' to this command task or set 'command_warnings=False' in ansible.cfg to get rid
node1 | CHANGED | rc=0 >>
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sun, 22 Mar 2026 17:58:04 GMT
Content-Type: text/html
Content-Length: 612
Last-Modified: Sun, 22 Mar 2026 17:28:23 GMT
Connection: keep-alive
ETag: "69c026b7-264"
Accept-Ranges: bytes  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0   612    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
```